



Proje Önerisi: Çok İş Parçacıklı Işın İzleme Motoru

Sistem Programlama Dersi Dönem Projesi

Enes F. Erkmén
220309007
Bilgisayar Mühendisliği

4 Nisan 2026

Özet

Bu proje, C++ programlama dili ve POSIX thread kütüphanesi (`std::thread`) kullanılarak Ubuntu işletim sistemi üzerinde gerçek zamanlı çalışan çok iş parçacıklı (*multi-threaded*) bir ışın izleme (*ray tracing*) motoru geliştirilmesini kapsamaktadır. Işın izleme, her pikselin bağımsız biçimde hesaplanabilmesi sayesinde paralel programlama için ideal bir uygulama alanı sunmaktadır. Proje kapsamında iş parçacığı sayısının render süresi, CPU kullanımı ve sistem yükü üzerindeki etkisi; `perf`, `valgrind`, `gprof` ve derleyici optimizasyon bayrakları (`-O0`, `-O2`, `-O3`) kullanılarak analiz edilecektir.

1 Giriş

Çok iş parçacıklı programlama, modern işlemcilerin çok çekirdekli mimarisinden tam verim elde edebilmek için kritik bir beceridir. Gerçek dünyada render motorları (Blender, V-Ray, Unreal Engine), bilimsel simülasyonlar ve görüntü işleme uygulamaları bu yaklaşımı yoğun biçimde kullanmaktadır.

Işın izleme algoritması, sanal bir kameradan sahneye fırlatılan ışınların nesnelerle kesişimini hesaplayarak fotorealistik görüntüler üretir. Temel prensip, her piksel için bağımsız bir hesaplama yapılmasıdır; bu da algoritmayı *embarrassingly parallel* kategorisine sokar. Yani her iş parçacığı diğerlerinden bağımsız çalışabilir ve senkronizasyon maliyeti minimuma indirilebilir.

Bu proje, söz konusu algoritmayı gerçek bir uygulama olarak hayata geçirirken sistem programlama açısından şu soruları yanıtlamayı hedefler:

- İş parçacığı sayısı artırıldıkça render süresi nasıl değişir?
- Thread pool mimarisi ile naif thread yaklaşımı arasındaki fark nedir?
- Derleyici optimizasyon seviyeleri performansı ne ölçüde etkiler?
- False sharing, cache miss gibi düşük seviyeli sorunlar nasıl tespit edilir ve giderilir?

2 Prosedür

Proje aşağıdaki adımlar izlenerek gerçekleştirilecektir:

1. **Temel Matematik Kütüphanesi:** 3 boyutlu vektör işlemleri (toplama, çarpma, iç çarpım, normalizasyon) için **Vec3** sınıfı yazılacaktır. Harici bağımlılık kullanılmayacaktır.
2. **Işın-Nesne Kesişim Hesapları:** Küre ve düzlem geometrileri için ışın-kesişim denklemleri implemente edilecektir. Sahnede birden fazla nesne, materyal (mat/diffuse ve yansıtıcı/reflective) ve nokta ışık kaynağı desteklenecektir.
3. **Tek İş Parçacıklı Temel Sürüm:** Önce tek thread ile çalışan referans bir render motoru tamamlanacak ve doğruluğu görsel olarak doğrulanacaktır.

4. **Thread Pool Mimarisi:** Ekran, eşit boyutlu tile'lara (örn. 64×64 piksel) bölünecektir. Bir iş kuyruğu (*task queue*) oluşturulacak; her iş parçacığı kuyruktan tile olarak işleyecektir. `std::mutex` ve `std::condition_variable` ile senkronizasyon sağlanacaktır.
5. **Performans Analizi:** Aynı sahne 1, 2, 4, 8 ve 16 iş parçacığıyla render edilecek; her konfigürasyon için süre, CPU kullanımı ve sistem yükü ölçülecektir.
6. **Araç Bazlı Analiz:** `perf stat`, `valgrind --tool=helgrind`, `-fsanitize=thread` ve `gprof` ile thread güvenliği, bellek sızıntısı ve darboğazlar incelenecektir.
7. **Çıktı ve Raporlama:** Render sonuçları PPM formatında kaydedilecek, ölçüm verileri grafiklerle raporlanacaktır.

3 Teori

3.1 Işın İzleme Algoritması

Işın izleme, her piksel (i, j) için kameradan geçen bir $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ ışınını sahneye fırlatır. Işının bir küreyle kesişimi şu denklemle belirlenir:

$$|\mathbf{o} + t\mathbf{d} - \mathbf{c}|^2 = R^2 \quad (1)$$

Bu denklem t cinsinden bir ikinci dereceden denklem verir. Gerçek kök varsa kesişim gerçekleşmiştir ve t_{min} değeri en yakın yüzeyi verir. Bulunan kesişim noktasında yüzey normali, ışık kaynağına olan yön ve materyal özellikleri kullanılarak Phong aydınlatma modeline göre piksel rengi hesaplanır.

3.2 Paralel Hesaplama ve Thread Pool

İş parçacığı havuzu (*thread pool*) modeli, her görev için yeni bir thread oluşturmanın getirdiği işletim sistemi maliyetini ortadan kaldırır. N adet thread sabit tutulur; işler bir kuyruktan dinamik olarak atanır. Bu yapı:

- Thread oluşturma/yok etme maliyetini sıfırlar,
- Yük dengelemeyi (*load balancing*) otomatik sağlar,

- Tile boyutu ile cache lokalitesi arasında optimize edilebilir bir denge sunar.

3.3 Beklenen Performans Davranışı

Amdahl Yasası'na göre, p paralel çalışan kısım oranı ve N iş parçacığı sayısı için teorik hızlanma:

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}} \quad (2)$$

Işın izleme hesaplamalarının büyük çoğunluğu paralel yürütülebildiğinden $p \approx 1$ beklenmekte; ancak thread senkronizasyon maliyeti, false sharing ve işletim sistemi zamanlaması nedeniyle ideal hızlanmadan sapma gözlemlenmesi öngörülmektedir. Bu sapmaların ölçülmesi ve açıklanması projenin temel analiz katkısı oluşturmaktadır.

Kaynaklar

- [1] P. Shirley, *Ray Tracing in One Weekend*, 2020. [Çevrimiçi]. Erişim: <https://raytracing.github.io>
- [2] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *AFIPS Conf. Proc.*, 1967.
- [3] cppreference.com, *std::thread*, *std::mutex*, *std::condition_variable*, 2024.